TRƯỜNG ĐẠI HỌC .....
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO ĐỒ ÁN MÔN HỌC

OSG203 — Hệ điều hành Linux

LINUXADMIN TOOLKIT

Bộ công cụ quản trị máy chủ bằng Bash cho doanh nghiệp TechNova Solutions

---

Đề tài 3: *Bash Scripting cho System Administration*

Giảng viên hướng dẫn: .....

Lớp: .....

Sinh viên thực hiện: Nguyễn Văn A — MSSV .....
Trần Thị B — MSSV .....
Lê Văn C — MSSV .....

TP. Hồ Chí Minh, tháng 10/2026

Tóm tắt

Báo cáo trình bày quá trình phân tích, thiết kế, hiện thực và kiểm thử **LinuxAdmin Toolkit**. Đây là bộ công cụ quản trị máy chủ Linux viết hoàn toàn bằng Bash, phục vụ một kịch bản doanh nghiệp giả định: công ty **TechNova Solutions** có một máy chủ Ubuntu Server tập trung (`technova-core-srv`) cho toàn bộ lập trình viên, data analyst và tester SSH vào làm việc. Tech Lead của công ty làm việc từ xa và chỉ dùng terminal qua SSH để điều hành hệ thống.

Toolkit gom các công việc hằng ngày của Tech Lead vào **5 chức năng chính**:

1. **users** — quản lý nhân viên và phân quyền;
2. **monitor** — giám sát tài nguyên và hoạt động của nhân viên;
3. **alert** — cảnh báo sự cố tức thời;
4. **backup** — sao lưu tài sản số;
5. **scheduler** — tự động hoá bảo dưỡng.

Hệ thống có **hai chế độ làm việc** chuyển qua lại được: giao diện menu trong terminal (TUI, dùng `dialog`/`whiptail`) và dòng lệnh (CLI). Hai chế độ dùng chung một bộ chức năng, vì mỗi thao tác trên giao diện thực chất gọi đúng một lệnh CLI tương ứng.

Sản phẩm gồm khoảng **3.850 dòng Bash** với **60 action**. Toolkit được đóng gói thành image Docker và gói `.tar.gz`, kèm bộ kiểm thử tự động. Bộ kiểm thử đạt **55/55 test PASS** trên Ubuntu 24.04, và mã nguồn qua kiểm tra tĩnh **shellcheck** không còn cảnh báo.

Từ khoá: Bash scripting, system administration, cron, sao lưu, giám sát, cảnh báo, TUI, Docker.

Mục lục

Tóm tắt **1**

1 Giới thiệu **5**
1.1 Bối cảnh doanh nghiệp . . . 5
1.2 Nhân vật chính và bài toán . . . 5
1.3 Mục tiêu và phạm vi . . . 5
1.4 Cấu trúc báo cáo . . . 6

2 Cơ sở lý thuyết **7**
2.1 Bash scripting . . . 7
2.1.1 Cấu trúc một script chuẩn . . . 7
2.1.2 Các kỹ thuật được sử dụng . . . 8
2.2 Người dùng, nhóm và quyền truy cập . . . 8
2.3 Tiến trình và tín hiệu . . . 9
2.4 Lập lịch tác vụ . . . 9
2.5 Sao lưu . . . 9
2.6 Giám sát và nhật ký hệ thống . . . 9
2.7 Giao diện menu trong terminal . . . 10

3 Phân tích và thiết kế **11**
3.1 Yêu cầu chức năng . . . 11
3.2 Yêu cầu phi chức năng . . . 11
3.3 Kiến trúc tổng thể . . . 12
3.4 Cơ chế “registry”: khai báo chức năng một lần . . . 12
3.5 Chế độ chạy thử (dry-run) . . . 13
3.6 Tổ chức mã nguồn và cấu hình . . . 13

4 Hiện thực **15**
4.1 Giao diện và chế độ làm việc . . . 15
4.2 Chức năng 1 — users: quản lý nhân viên và phân quyền . . . 16
4.3 Chức năng 2 — monitor: giám sát tài nguyên và nhân viên . . . 17
4.4 Chức năng 3 — alert: cảnh báo sự cố tức thời . . . 18
4.5 Chức năng 4 — backup: sao lưu tài sản số . . . 19
4.6 Chức năng 5 — scheduler: tự động hoá bảo dưỡng . . . 20
4.7 Đóng gói và triển khai . . . 20

5 Kiểm thử và đánh giá **22**
5.1 Môi trường kiểm thử 22
5.2 Kịch bản kiểm thử tự động 22
5.3 Đánh giá lợi ích của tự động hoá 24
5.4 Hạn chế 24

6 Kết luận và hướng phát triển **25**
6.1 Kết luận 25
6.2 Hướng phát triển 25

A Hướng dẫn cài đặt nhanh **26**

B Danh sách lệnh **27**

Danh sách hình vẽ

3.1 Kiến trúc LinuxAdmin Toolkit 12

4.1 Menu chính của giao diện TUI khi SSH vào technova-core-srv 15
4.2 Giao diện hỏi tham số bằng hộp thoại (monitor → hogs) 16
4.3 Kiểm chứng phân quyền theo phòng ban 17
4.4 Quy trình nhân viên nghỉ việc (offboard) 17
4.5 Phát hiện Data Analyst `cuonglv` treo `train_model.py` chiếm 99% CPU 18
4.6 alert check phát hiện process ngốn tài nguyên 19
4.7 Sao lưu và kiểm tra toàn vẹn bản sao 19
4.8 Cài lịch tự động và giải thích biểu thức cron bằng tiếng Việt 20

Danh sách bảng

2.1 Kỹ thuật Bash và vị trí sử dụng trong toolkit 8

3.1 Ánh xạ nhu cầu của Tech Lead sang 5 chức năng 11

4.1 Lịch tự động mặc định 20

5.1 Kết quả kiểm thử tự động 23

5.2 So sánh thao tác thủ công và dùng toolkit (ước tính của nhóm) 24

CHƯƠNG 1

Giới thiệu

1.1 Bối cảnh doanh nghiệp

TechNova Solutions là công ty công nghệ chuyên phát triển phần mềm và phân tích dữ liệu. Hạ tầng của công ty xoay quanh một **máy chủ Linux Ubuntu Server tập trung** tên `technova-core-srv`, đặt tại trụ sở chính. Toàn bộ nhân sự đều SSH vào máy chủ này để làm việc:

- **Lập trình viên** (nhóm `dev`) viết và commit code;
- **Data Analyst** (nhóm `data`) chạy các tác vụ huấn luyện mô hình dữ liệu;
- **Tester** (nhóm `qa`) chạy kiểm thử.

1.2 Nhân vật chính và bài toán

Anh Hưng là Giám đốc Kỹ thuật (Tech Lead/Sysadmin). Anh làm việc từ xa và **chỉ dùng terminal qua SSH** để điều hành hạ tầng. Công việc hằng ngày của anh gồm bốn nhóm:

1. Cấp quyền và quản lý nhân viên làm việc: tài khoản, phòng ban, thư mục dự án, quyền sudo, thủ tục nghỉ việc.
2. Giám sát tài nguyên máy chủ: ai đang đăng nhập và làm gì, có ai treo process ngốn RAM/CPU không.
3. Nhận cảnh báo tức thời khi hệ thống gặp sự cố, bất kể ngày hay đêm.
4. Tự động hoá các tác vụ bảo dưỡng hằng ngày và sao lưu an toàn tài sản số của công ty.

Nếu làm tay, mỗi việc trên gồm hàng chục lệnh rời rạc (`useradd`, `chmod`, `ps`, `kill`, `tar`, `crontab`...), dễ sai sót và khó lặp lại. Bài toán đặt ra là **tự động hoá toàn bộ các công việc này bằng Bash script**.

1.3 Mục tiêu và phạm vi

- Xây dựng một bộ công cụ **viết hoàn toàn bằng Bash**, chạy được trên Ubuntu/Debian (và tương thích RHEL/Rocky).
- Đáp ứng đủ **5 chức năng chính** tương ứng với nhu cầu của Tech Lead.
- Dùng tốt qua SSH với **hai chế độ**: giao diện menu (TUI) và dòng lệnh (CLI), chuyển qua lại được.

- **An toàn**: mọi thao tác thay đổi hệ thống đều hỏi xác nhận và có chế độ chạy thử (*dry-run*).
- **Dễ triển khai**: cài một lệnh trên máy thật, hoặc chạy bằng Docker.

Ngoài phạm vi: quản lý nhiều máy chủ cùng lúc, giao diện web, tích hợp hệ thống giám sát chuyên dụng (Prometheus, Zabbix).

1.4 Cấu trúc báo cáo

- **Chương 2**: cơ sở lý thuyết về Bash và quản trị Linux.
- **Chương 3**: phân tích yêu cầu và thiết kế hệ thống.
- **Chương 4**: hiện thực từng chức năng.
- **Chương 5**: kiểm thử và đánh giá.
- **Chương 6**: kết luận và hướng phát triển.

CHƯƠNG 2

Cơ sở lý thuyết

2.1 Bash scripting

Bash (Bourne Again SHell) là shell mặc định trên hầu hết các bản phân phối Linux. Một Bash script là tệp văn bản chứa các lệnh shell được thực thi tuần tự. Bash có đủ cấu trúc của một ngôn ngữ lập trình: biến, điều kiện, vòng lặp, hàm và mảng.

2.1.1 Cấu trúc một script chuẩn

```
#!/usr/bin/env bash          # shebang: chọn trình thông dịch
set -Eeuo pipefail           # chế độ "nghiêm ngặt"
trap 'on_error $LINENO' ERR  # bắt lỗi, báo số dòng
trap 'cleanup' EXIT          # dọn file tạm khi thoát
main() { ... }               # chương trình chính trong hàm
main "$@"
```

Ý nghĩa các tuỳ chọn `set`:

- `-e`: dừng ngay khi một lệnh lỗi;
- `-u`: báo lỗi khi dùng biến chưa khai báo;
- `-o pipefail`: một pipeline bị coi là lỗi nếu bất kỳ lệnh nào trong đó lỗi;
- `-E`: bẫy `ERR` có hiệu lực cả bên trong hàm.

2.1.2 Các kỹ thuật được sử dụng

Bảng 2.1: Kỹ thuật Bash và vị trí sử dụng trong toolkit

Kỹ thuật	Ứng dụng trong toolkit
Hàm, biến `local`, mã thoát	Mỗi chức năng là một hàm `<module>_<action>`
`getopts` + tuỳ chọn dài	Phân tích `--tui`, `--cli`, `--dry-run`, `--yes`
Mảng kết hợp (`declare -A`)	"Registry" lưu module, action, tham số
Mở rộng tham số	`${var:-mặc định}`, `${var//a/b}`, `${var%%.*}`
Biểu thức chính quy `[[ =~ ]]`	Kiểm tra username, PID, biểu thức cron
Here-doc, `< <(...)`, `mapfile`	Đọc CSV nhân viên, crontab, danh sách tiến trình
`awk`	Tổng hợp CPU/RAM theo người dùng, tách trường log
`trap ERR/EXIT/INT`	Báo lỗi, dọn file tạm, xử lý Ctrl+C
Tiến trình nền, `wait`, `timeout`	Tạo tải giả lập trong `alert stress`
`flock`	Chống hai tiến trình sao lưu chạy cùng lúc
`logger`	Ghi sự kiện vào journald/syslog

2.2 Người dùng, nhóm và quyền truy cập

Thông tin tài khoản được lưu ở các tệp sau:

- `/etc/passwd`: tên đăng nhập, UID, GID, thư mục home, shell;
- `/etc/shadow`: mật khẩu đã băm và chính sách hết hạn (chỉ root đọc được);
- `/etc/group`: danh sách nhóm và thành viên.

Mỗi tệp và thư mục có quyền **rwx** cho ba đối tượng: chủ sở hữu (user), nhóm (group) và người khác (other). Quyền thường được biểu diễn bằng số bát phân, trong đó $r = 4$, $w = 2$, $x = 1$. Ngoài ra có ba bit đặc biệt:

- **SUID** (4): chương trình chạy với quyền của chủ sở hữu tệp;
- **SGID** (2): trên thư mục, tệp mới tạo bên trong *kế thừa nhóm* của thư mục. Toolkit dùng bit này cho thư mục phòng ban, với quyền `2770`;
- **Sticky bit** (1): trong thư mục ai cũng ghi được, mỗi người chỉ xoá được tệp của chính mình. Toolkit dùng bit này cho thư mục chung, với quyền `1777`.

Quyền quản trị được cấp qua nhóm `sudo` (Debian/Ubuntu) hoặc `wheel` (RHEL). Chính sách mật khẩu được điều khiển bằng `/etc/login.defs`, lệnh `chage` và module PAM `pam_pwquality`.

2.3 Tiến trình và tín hiệu

Mỗi tiến trình có một PID và một tiến trình cha (PPID). Kernel cung cấp thông tin về từng tiến trình qua hệ thống tệp ảo `/proc/<pid>/`. Một số công cụ quan trọng:

- `ps`, `top`: liệt kê tiến trình;
- `w`, `who`, `last`: phiên đăng nhập và lịch sử đăng nhập;
- `nice`, `renice`: đổi độ ưu tiên, từ -20 (cao nhất) đến 19 (thấp nhất);
- `kill`: gửi tín hiệu tới tiến trình.

Hai tín hiệu hay dùng nhất là **SIGTERM** (15), yêu cầu kết thúc ”lịch sự” để chương trình kịp dọn dẹp, và **SIGKILL** (9), buộc dừng ngay lập tức.

2.4 Lập lịch tác vụ

cron chạy tác vụ định kỳ theo biểu thức gồm 5 trường: *phút, giờ, ngày, tháng, thứ*. Ví dụ, `30 3 * * *` nghĩa là chạy lúc 03:30 mỗi ngày, còn `*/5 * * * *` nghĩa là chạy mỗi 5 phút.

at hẹn chạy một lần, ví dụ `at now + 2 minutes`. **systemd timer** là cách lập lịch hiện đại: có ghi log vào journald và có thể chạy bù nếu máy tắt đúng giờ hẹn (`Persistent=true`).

2.5 Sao lưu

Toolkit áp dụng các nguyên tắc sao lưu sau:

- nén bằng `tar`, giữ nguyên quyền và chủ sở hữu của tệp;
- tạo **checksum SHA-256** để phát hiện bản sao bị hỏng hoặc bị sửa đổi;
- **xoay vòng**: chỉ giữ N bản mới nhất;
- lưu thêm một bản **ngoài máy chủ** bằng `rsync` qua SSH, theo tinh thần quy tắc 3-2-1;
- định kỳ **kiểm tra khả năng khôi phục**.

2.6 Giám sát và nhật ký hệ thống

Các chỉ số tài nguyên được đọc trực tiếp từ kernel:

- mức sử dụng CPU: tính từ `/proc/stat`, bằng hiệu số giữa hai lần lấy mẫu;
- bộ nhớ: `/proc/meminfo`, dùng chỉ số *MemAvailable*;
- tải hệ thống: `/proc/loadavg`;
- dung lượng đĩa: lệnh `df`.

Nhật ký hệ thống do **journald** thu thập (`journalctl`) và có thể chuyển tiếp sang **rsyslog** để lưu thành tệp văn bản trong `/var/log`.

2.7 Giao diện menu trong terminal

`dialog` và `whiptail` vẽ hộp thoại (menu, ô nhập, hộp xác nhận, hộp xem văn bản) ngay trong terminal bằng thư viện ncurses/newt. Nhờ vậy giao diện hoạt động tốt qua SSH, không cần môi trường đồ hoạ. Lựa chọn của người dùng được trả về qua luồng lỗi chuẩn (stderr).

CHƯƠNG 3

Phân tích và thiết kế

3.1 Yêu cầu chức năng

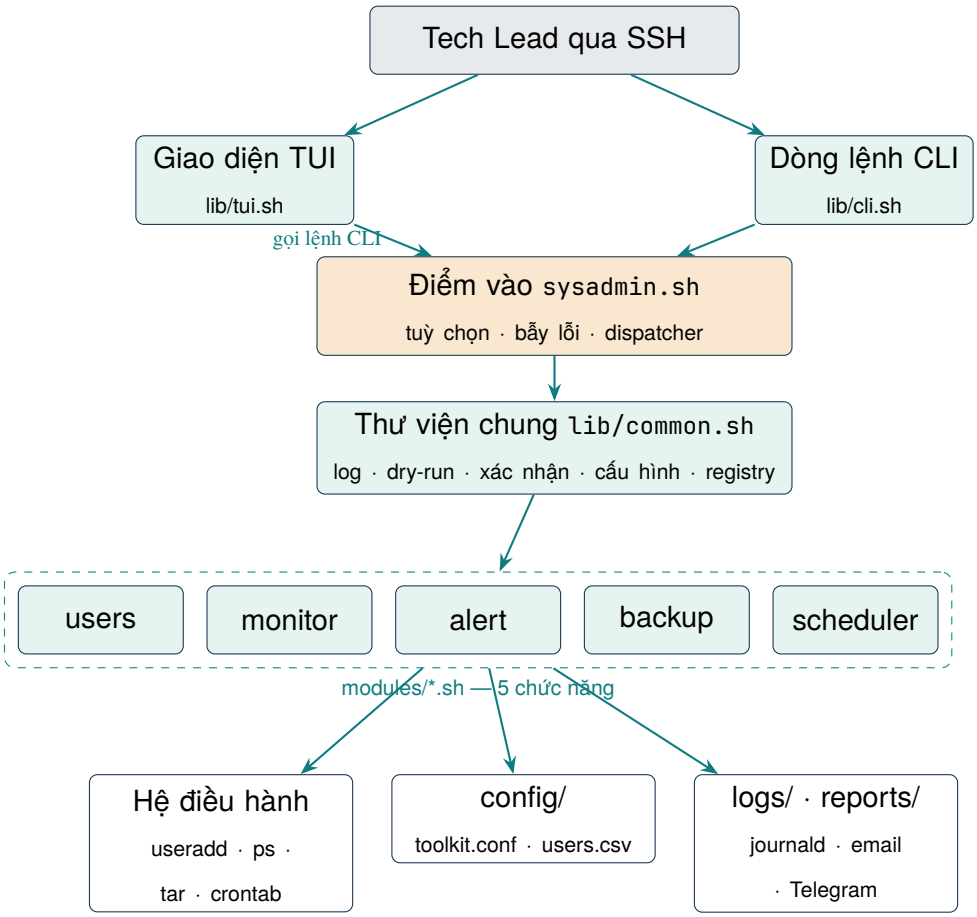
Bảng 3.1: Ánh xạ nhu cầu của Tech Lead sang 5 chức năng

#	Nhu cầu	Chức năng	Các action chính
1	Cấp quyền, quản lý nhân viên	`users`	create, setup-dirs, sudo, lock/unlock, passwd, kick, **offboard**, audit, demo
2	Ai đang làm gì, ai treo process	`monitor`	**who**, **usage**, top, **hogs**, renice, kill, troubleshoot, report, watch
3	Cảnh báo tức thời ngày đêm	`alert`	**check**, setup-telegram, setup-mail, test, set-threshold, stress
4	Sao lưu tài sản số	`backup`	**run**, list, verify, restore, rotate, history
5	Tự động hoá bảo dưỡng	`scheduler`	**install**, **maintenance**, add, explain, at, timers-install

3.2 Yêu cầu phi chức năng

- **Vận hành từ xa**: mọi chức năng dùng được qua SSH, không cần môi trường đồ hoạ.
- **An toàn**: thao tác thay đổi hệ thống phải được xác nhận và có chế độ chạy thử. Toolkit từ chối thao tác lên tài khoản hệ thống (UID $<$ 1000) và PID 1.
- **Lặp lại được (idempotent)**: chạy lại một lệnh không gây trùng lặp, ví dụ tạo lại user đã có hoặc cài lại lịch cron.
- **Truy vết**: mọi lệnh thay đổi hệ thống đều được ghi log ra tệp và journald.
- **Di động**: chạy trên Debian/Ubuntu và RHEL/Rocky; có bản Docker.

3.3 Kiến trúc tổng thể



Hình 3.1: Kiến trúc LinuxAdmin Toolkit

Điểm mấu chốt của thiết kế là **giao diện chỉ là lớp vỏ**. Khi người dùng chọn một chức năng trên menu TUI, toolkit hỏi tham số qua hộp thoại, sau đó gọi lại chính nó dưới dạng lệnh CLI trong một tiến trình riêng, ví dụ `sysadmin monitor hogs 90 30`. Kết quả được hiển thị kèm lệnh tương đương. Nhờ vậy hai chế độ luôn cho kết quả giống hệt nhau, và người học giao diện cũng đồng thời học được dòng lệnh.

3.4 Cơ chế "registry": khai báo chức năng một lần

Mỗi module tự khai báo các action của mình kèm cờ và danh sách tham số. Từ khai báo đó, trang trợ giúp CLI, menu CLI và menu TUI đều được **sinh tự động**:

```
register_module monitor 2 "Giám sát tài nguyên & hoạt động nhân viên"
register_action hogs "" "Phát hiện process treo ngốn tài nguyên" \
                "Ngưỡng CPU (%)=$HOG_CPU" "Ngưỡng RAM (%)=$HOG_MEM"
```

```
register_action kill "rd" "Dừng tiến trình gây sự cố" \
                "PID=@_pick_pid" "Tín hiệu=TERM|KILL|HUP|STOP|CONT"
```

Quy ước khai báo:

- cờ `r`: action cần quyền root;
- cờ `d`: action thay đổi hệ thống, phải hỏi xác nhận;
- tham số dạng `"Lời nhắc=mặc định"`: có giá trị mặc định;
- tham số dạng `a|b|c`: chọn một trong danh sách;
- tham số dạng `@hàm`: chọn từ danh sách động do hàm sinh ra, ví dụ danh sách tiến trình.

Hàm thực thi tương ứng có tên `monitor_hogs`, `monitor_kill`. Muốn thêm chức năng mới, chỉ cần tạo thêm một tệp trong `modules/`.

3.5 Chế độ chạy thử (dry-run)

Mọi lệnh làm thay đổi hệ thống đều được bọc qua hàm `run`. Ở chế độ `--dry-run`, hàm này chỉ in lệnh ra mà không thực thi. Ở chế độ thường, nó ghi log trước khi chạy và báo lỗi rõ ràng nếu lệnh thất bại:

```
run() {
  if [[ $DRY_RUN == 1 ]]; then
    printf '[DRY-RUN] %s\n' "$(shell_quote "$@")"; return 0
  fi
  _log_write CMD "$(shell_quote "$@")"
  "$@" && return 0
  local rc=$?
  log_error "Lệnh thất bại (mã $rc): $(shell_quote "$@")"
  return "$rc"
}
```

3.6 Tổ chức mã nguồn và cấu hình

```
sysadmin-toolkit/
├── sysadmin.sh            điểm vào: getopts, chọn chế độ, trap
├── install.sh             cài vào /opt, tạo lệnh "sysadmin"
├── lib/   common.sh · cli.sh · tui.sh
├── modules/   users · monitor · alert · backup · scheduler
├── config/    toolkit.conf · users.csv
```

```
├── systemd/   mẫu .service/.timer
├── tests/demo-test.sh     kiểm thử tự động 55 bước
└── Dockerfile · docker-compose.yml · package.sh
```

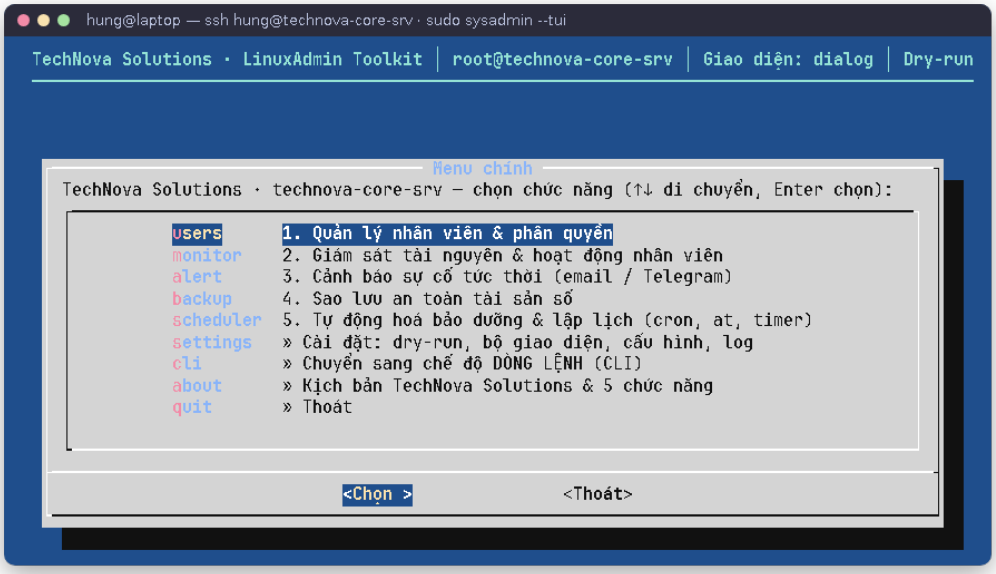
Tệp `config/toolkit.conf` chứa các tham số vận hành: tên công ty, danh sách phòng ban, ngưỡng cảnh báo, thư mục sao lưu, lịch cron, kênh email/Telegram. Các tham số này được nạp lại ở mỗi lần chạy, nên sửa xong là có hiệu lực ngay.

CHƯƠNG 4

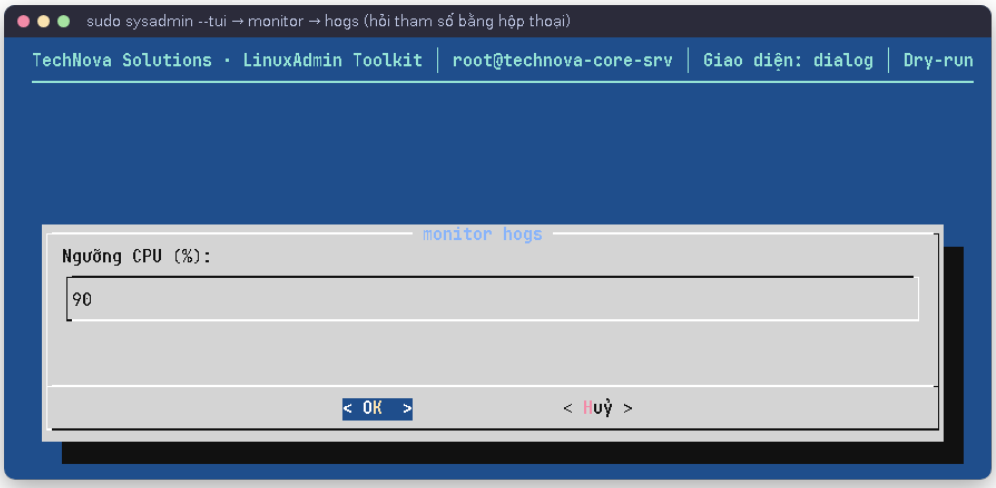
Hiện thực

4.1 Giao diện và chế độ làm việc

Khi chạy `sysadmin` không kèm tham số, toolkit cho người dùng chọn chế độ. Chế độ TUI hiển thị menu 5 chức năng (Hình 4.1). Thanh tiêu đề luôn cho biết người dùng, máy chủ, bộ giao diện đang dùng và trạng thái dry-run. Với các action có tham số, giao diện hỏi lần lượt bằng hộp thoại (Hình 4.2). Với thao tác thay đổi hệ thống, giao diện cho chọn *Chạy thật / Chạy thử / Huỷ*.



Hình 4.1: Menu chính của giao diện TUI khi SSH vào technova-core-srv



Hình 4.2: Giao diện hỏi tham số bằng hộp thoại (monitor → hogs)

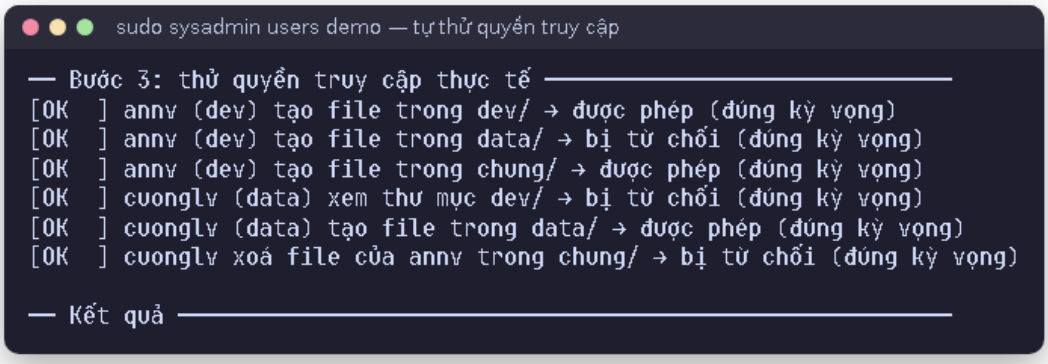
4.2 Chức năng 1 — users: quản lý nhân viên và phân quyền

Danh sách nhân viên được khai báo trong `config/users.csv`. Mỗi dòng gồm username, họ tên, phòng ban, shell và quyền sudo. Lệnh `users create` xử lý từng nhân viên theo các bước:

1. kiểm tra tên hợp lệ bằng biểu thức chính quy;
2. bỏ qua nếu tài khoản đã tồn tại;
3. tạo nhóm phòng ban nếu chưa có;
4. gọi `useradd` với nhóm phụ (thêm nhóm sudo/wheel nếu được cấp quyền);
5. đặt mật khẩu ngẫu nhiên 12 ký tự qua `chpasswd`;
6. ép đổi mật khẩu ở lần đăng nhập đầu và áp chính sách hết hạn (`chage -d 0 -M 90 -m 1 -W 7`).

Mật khẩu tạm được lưu vào một tệp CSV có quyền `600`.

Lệnh `users setup-dirs` tạo cây thư mục dự án: `/srv/technova/{dev,data,qa}` với quyền `2770` và bit SGID, cùng thư mục `chung/` với quyền `1777` và sticky bit. Lệnh `users demo` tự động **thử quyền truy cập thực tế** dưới danh nghĩa từng nhân viên bằng `runuser` (Hình 4.3).

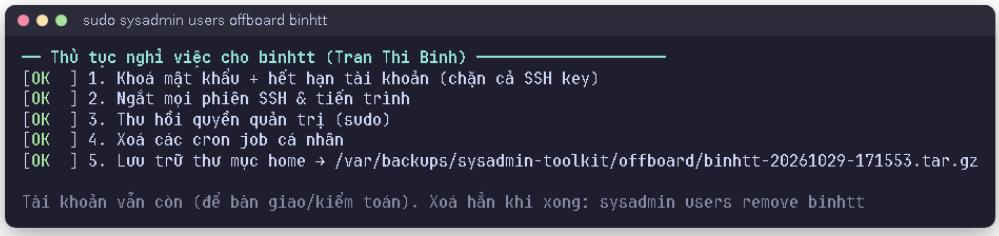


Hình 4.3: Kiểm chứng phân quyền theo phòng ban

Khi nhân viên nghỉ việc, lệnh `users offboard` thực hiện liền một quy trình 5 bước (Hình 4.4):

1. khoá mật khẩu và đặt ngày hết hạn tài khoản (`usermod -L -e 1`), chặn cả đăng nhập bằng SSH key;
2. ngắt mọi phiên làm việc (`loginctl terminate-user`, `pkill -KILL -u`);
3. thu hồi quyền sudo;
4. xoá crontab cá nhân;
5. nén thư mục home để lưu trữ.

Lệnh `users audit` kiểm tra bảo mật trên 10 nhóm tiêu chí: tài khoản UID 0, mật khẩu rỗng hoặc không hết hạn, quyền của `/etc/shadow` và `/etc/sudoers`, quy tắc NOPASSWD, quyền thư mục home, tệp ai cũng ghi được, tệp SUID và cấu hình SSH.



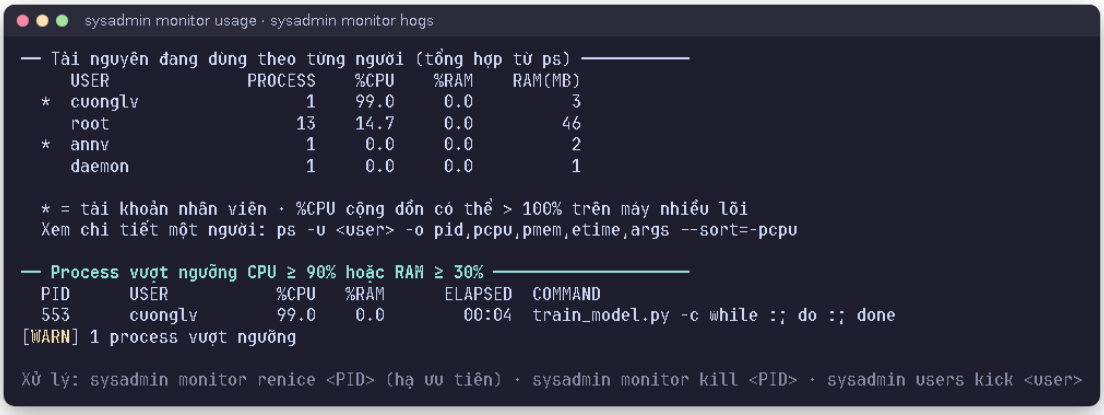
Hình 4.4: Quy trình nhân viên nghỉ việc (offboard)

4.3 Chức năng 2 — monitor: giám sát tài nguyên và nhân viên

- `monitor who` trả lời câu hỏi *ai đang đăng nhập, từ IP nào, đang chạy lệnh gì* (dùng `w`, `last`, `lastb`).
- `monitor usage` cộng dồn CPU/RAM **theo từng nhân viên** bằng `awk`:

```
ps -eo user:20=,uid=,pcpu=,pmem=,rss= | awk -v min="$UID_MIN" '
  { u=$1; p[u]++; c[u]+=$3; m[u]+=$4; r[u]+=$5; id[u]=$2 }
  END { for (u in p) printf "%.1f\t%s %d %.1f %.1f %.0f\n",
        c[u], u, p[u], c[u], m[u], r[u]/1024 }' | sort -rn
```

- `monitor hogs` lọc các process vượt ngưỡng (mặc định $\geq$ 90% CPU hoặc $\geq$ 30% RAM) và hiển thị dòng lệnh đầy đủ, giúp nhận ra ngay ai đang treo process gì (Hình 4.5).
- Để xử lý, Tech Lead có thể hạ độ ưu tiên bằng `monitor renice` (mềm mỏng) hoặc dừng tiến trình bằng `monitor kill`. Lệnh kill có xác nhận và từ chối PID 1.



Hình 4.5: Phát hiện Data Analyst `cuonglv` treo `train_model.py` chiếm 99% CPU

Ngoài ra, module còn có:

- `troubleshoot`: quy trình chẩn đoán 10 bước (tải, CPU, iowait, RAM/swap, OOM, đĩa/inode, dịch vụ lỗi, mạng/DNS, log lỗi, NTP/zombie), kèm gợi ý xử lý;
- `report`: xuất báo cáo HTML có biểu đồ CPU/RAM theo thời gian;
- `watch`: bảng theo dõi thời gian thực.

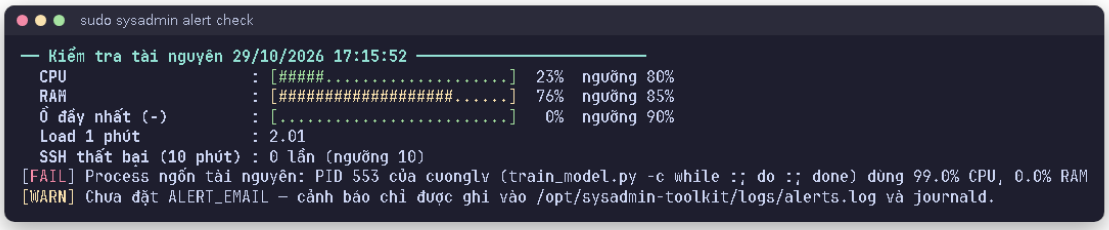
4.4 Chức năng 3 — alert: cảnh báo sự cố tức thời

`alert check` được cron gọi mỗi 5 phút và thực hiện tuần tự:

1. đo CPU (lấy mẫu `/proc/stat`), RAM, ổ đĩa, rồi ghi vào `metrics.csv`;
2. phát hiện process ngốn tài nguyên, kèm tên nhân viên đang chạy;
3. phát hiện dịch vụ systemd bị lỗi;
4. đếm số lần đăng nhập SSH thất bại trong 10 phút (dấu hiệu bị dò mật khẩu);

5. nếu có sự cố, gửi cảnh báo qua **email** (msmtp/mail) và **Telegram** (gọi Bot API bằng `curl`).

Có **thời gian chờ (cooldown) 30 phút** để không gửi trùng lặp lúc nửa đêm. Khi hết sự cố, toolkit gửi tin **"đã ổn định"**. Tiêu đề email tiếng Việt được mã hoá theo RFC 2047 (`=?UTF-8?B? ... ?=`).



Hình 4.6: alert check phát hiện process ngốn tài nguyên

4.5 Chức năng 4 — backup: sao lưu tài sản số

Dữ liệu được sao lưu mặc định gồm `/srv/technova` (dự án), `/home` (nhân viên) và `/etc` (cấu hình). Một lần sao lưu đi qua các bước sau:

1. khoá `flock` để chống chạy trùng;
2. nén bằng `tar -czp` (bỏ qua tệp không đọc được thay vì làm hỏng cả bản sao);
3. kiểm tra tệp nén (`gzip -t`);
4. tạo checksum SHA-256;
5. xoay vòng, giữ 7 bản mới nhất;
6. tuỳ chọn `rsync` bản sao ra máy khác;
7. ghi lịch sử, và gửi cảnh báo nếu thất bại.

```
exec 9>"$dest/.backup.lock"
flock -n 9 || die "Đang có một tiến trình sao lưu khác chạy."
tar -czpf "$file" --ignore-failed-read "${excl[@]}" -C / "${rel[@]}"
gzip -t "$file" && (cd "$dest" && sha256sum "$name" > "$name.sha256")
```



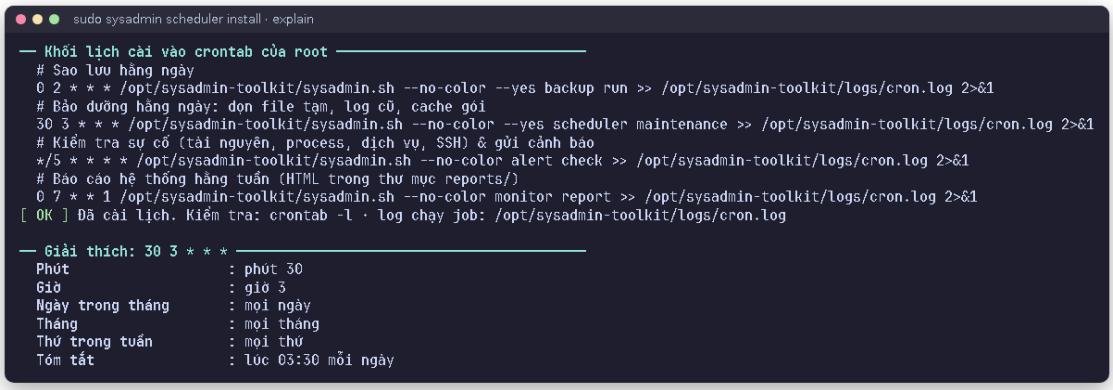
Hình 4.7: Sao lưu và kiểm tra toàn vẹn bản sao

4.6 Chức năng 5 — scheduler: tự động hoá bảo dưỡng

`scheduler install` ghi một **khối lịch riêng** vào crontab của root, nằm giữa hai dòng đánh dấu `>>> sysadmin-toolkit >>>`. Nhờ vậy, cài lại hay gỡ bỏ đều không ảnh hưởng các job khác của hệ thống.

Bảng 4.1: Lịch tự động mặc định

Biểu thức cron	Thời điểm	Tác vụ
`0 2 * * *`	02:00 hằng ngày	`backup run`
`30 3 * * *`	03:30 hằng ngày	`scheduler maintenance`: dọn /tmp, log journald cũ, cache gói, core dump, xoay vòng backup
`*/5 * * * *`	mỗi 5 phút	`alert check`
`0 7 * * 1`	07:00 thứ Hai	`monitor report`



Hình 4.8: Cài lịch tự động và giải thích biểu thức cron bằng tiếng Việt

Toolkit cũng có thể dùng systemd timer thay cho cron (`scheduler timers-install`), và hỗ trợ hẹn giờ chạy một lần bằng `at`.

4.7 Đóng gói và triển khai

- **Cài trên máy thật:** `sudo ./install.sh --deps` chép toolkit vào `/opt/sysadmin-toolkit` (chủ sở hữu root), tạo lệnh `sysadmin`, cài các gói phụ thuộc và bật tự hoàn thành lệnh bằng phím Tab.
- **Docker:** `docker compose up -d` dựng sẵn máy chủ `technova-core-srv` (Ubuntu 24.04, cron, at, SSH server). Tech Lead SSH vào cổng 2222 như với máy thật; bản sao lưu, cấu hình, báo cáo và log được giữ trong volume.

- **Gói nén:** `./package.sh` tạo tệp `.tar.gz` kèm checksum để mang sang máy Linux khác.

CHƯƠNG 5

Kiểm thử và đánh giá

5.1 Môi trường kiểm thử

- **Ubuntu 24.04** chạy trong container Docker, hostname `technova-core-srv`, quyền root, có `CAP_SYS_NICE`.
- **Kali Linux Rolling** (kernel 7.1, bash 5.3), tài khoản thường: chạy các chức năng chỉ đọc và chế độ dry-run.
- Kiểm tra tĩnh bằng **shellcheck**.

5.2 Kịch bản kiểm thử tự động

Script `tests/demo-test.sh` mô phỏng trọn "một ngày của anh Hưng" theo thứ tự: nhân viên mới vào làm → một nhân viên treo process → cảnh báo → sao lưu → tự động hoá → nhân viên nghỉ việc → dọn dẹp. Mỗi bước được kiểm chứng bằng điều kiện cụ thể trên hệ thống, ví dụ: thành viên nhóm, quyền thư mục, tiến trình còn sống hay không, nội dung crontab, nội dung tệp khôi phục.

Bảng 5.1: Kết quả kiểm thử tự động

Nhóm	Nội dung kiểm chứng	Số test	Kết quả
0. Nền tảng	Phiên bản, help sinh tự động, từ chối tham số sai, dry-run không thay đổi hệ thống	5	5/5
1. users	Tạo 7 nhân viên, phòng ban, sudo, ép đổi mật khẩu, tệp mật khẩu 600, idempotent, quyền 2770/1777, SGID, sticky, lock/unlock, audit	18	18/18
2. monitor	Mô phỏng process treo, usage, hogs, top, who/overview/troubleshoot, renice	6	6/6
3. alert	Phát hiện sự cố, ghi log, cooldown, kill, tin "đã ổn định", metrics.csv, đổi ngưỡng	7	7/7
4. backup	Tạo bản sao, SHA-256 khớp, chứa dữ liệu dự án, verify, khôi phục đúng nội dung, xoay vòng	6	6/6
5. scheduler	Khối cron đủ 4 job, job tuỳ chỉnh (thoát %), explain, từ chối cron sai, bảo dưỡng xoá tệp cũ, gỡ lịch	7	7/7
1'. Nghỉ việc	kick, offboard (khoá + lưu trữ home), từ chối offboard root	4	4/4
Dọn dẹp	Xoá nhân viên, thư mục, nhóm demo	2	2/2
Tổng		**55**	**55/55**

Chạy lại bộ kiểm thử bằng lệnh:

```
docker run --rm --cap-add SYS_NICE -v "$PWD/tests:/tests:ro" \
       technova/sysadmin-toolkit:1.0 bash /tests/demo-test.sh --yes
```

5.3 Đánh giá lợi ích của tự động hoá

Bảng 5.2: So sánh thao tác thủ công và dùng toolkit (ước tính của nhóm)

Công việc	Làm tay	Với toolkit
Tạo 7 nhân viên + phòng ban + thư mục	$\approx$ 50 lệnh, 20–30 phút, dễ sai quyền	2 lệnh, $<$ 10 giây, có kiểm chứng
Tìm ai đang treo process	`ps`/`top`, tự đối chiếu user	`monitor hogs`: một lệnh
Phát hiện sự cố lúc 2 giờ sáng	Chỉ biết khi nhân viên phản ánh	Cảnh báo trong $\leq$ 5 phút
Sao lưu hằng ngày	Dễ quên, không kiểm tra bản sao	Tự động, có checksum và xoay vòng
Nhân viên nghỉ việc	Dễ sót bước (sudo, crontab, phiên còn mở)	`users offboard`: 5 bước một lần

5.4 Hạn chế

- Chỉ quản lý một máy chủ; chưa có cơ chế quản lý tập trung nhiều máy.
- Bản Docker không có systemd, nên chức năng systemd timer và log journald bị hạn chế (vẫn dùng cron được).
- Gửi email phụ thuộc vào cấu hình SMTP bên ngoài (Gmail App Password); nhóm chưa kiểm thử gửi thư thật.
- Giao diện whiptail cần nhấn Tab để đóng hộp kết quả; dùng `dialog` sẽ thuận tiện hơn.

CHƯƠNG 6

Kết luận và hướng phát triển

6.1 Kết luận

Đồ án đã xây dựng thành công **LinuxAdmin Toolkit**. Bộ công cụ chứng minh rằng Bash scripting đủ sức tự động hoá toàn bộ công việc quản trị hằng ngày của một máy chủ doanh nghiệp, cụ thể:

- quản lý vòng đời nhân viên, từ tiếp nhận đến nghỉ việc;
- phân quyền theo phòng ban;
- giám sát và xử lý tiến trình gây sự cố;
- cảnh báo tức thời;
- sao lưu an toàn;
- bảo dưỡng định kỳ.

Thiết kế "giao diện là lớp vỏ của CLI" giúp toolkit vừa thân thiện qua SSH, vừa dễ dùng trong script và cron. Bộ kiểm thử tự động 55 bước xác nhận các chức năng hoạt động đúng.

6.2 Hướng phát triển

- Quản lý nhiều máy chủ qua SSH hoặc chuyển sang Ansible.
- Dashboard web đọc dữ liệu `metrics.csv` và báo cáo.
- Tích hợp xác thực hai lớp và theo dõi lệnh sudo của nhân viên (`auditd`).
- Đẩy bản sao lưu lên dịch vụ lưu trữ đám mây và mã hoá bản sao bằng GPG.

PHỤ LỤC A

Hướng dẫn cài đặt nhanh

```
# Cách A — Docker (máy chủ giả lập technova-core-srv)
git clone https://github.com/duylmaoxd/technova-sysadmin-toolkit.git
cd technova-sysadmin-toolkit && docker compose up -d --build
ssh -p 2222 hung@localhost        # rồi: sudo sysadmin --tui

# Cách B — cài trực tiếp trên Ubuntu Server
sudo ./install.sh --deps
sudo sysadmin scheduler install
```

Mã nguồn: `https://github.com/duylmaoxd/technova-sysadmin-toolkit`. Hướng dẫn chi tiết: tệp `HUONG-DAN-SU-DUNG.md`.

PHỤ LỤC B

Danh sách lệnh

Chức năng	Action
users	list, create, delete, add, remove, lock, unlock, passwd, sudo, policy, set-policy, audit, perm, shared-dir, setup-dirs, kick, offboard, demo, demo-clean
monitor	who, usage, top, hogs, renice, kill, overview, memory, disk, logs, auth, troubleshoot, report, watch, history
alert	check, status, test, setup-mail, setup-telegram, set-threshold, stress, history
backup	run, list, verify, restore, rotate, history
scheduler	list, install, maintenance, remove, add, explain, at, at-list, at-remove, log, timers-install, timers-remove

Tuỳ chọn chung: `--tui`, `--cli`, `--dry-run`, `--yes`, `--ascii`, `--no-color`, `--config`. Xem chi tiết: `sysadmin help <chức năng>`.